

Our final two modules

Domains: Images, Internet and Big Data

Practical skills: 2D and 3D lists, image manipulation, pass by reference, search, sort, recursion

Image Processing

Housekeeping

- things that are due for the rest of the semester
 - assignment 5
 - project
 - one final reflection
 - please come to the final

That's it! I'll keep all these things in mind for our remaining class time and try to help you make progress on project and exam prep.

Overview of content and reminder of learning outcomes (slide 1)

You should know:

- Modules!
 - How to create and use a module containing one's own defined functions
 - How to import and access the contents of packages and modules
 - How to import a module with a short name

Overview of content and reminder of learning outcomes (slide 2)

- Image processing and 2D list / 3D list manipulation:
 - How pixel colors are represented by RGB values
 - How to pass global variables into local scope
 - That you can to directly return a Boolean expression in a function without using if statement in the function (e.g. `return x > 10`)
 - How to access and modify a 2D image in the form of a list of lists (with 0s and 1s)
 - How to access and modify a 3d image in the form of a list of lists (with 3-item RGB lists)

- More from Image processing and 2D list / 3D list manipulation:
 - How to manipulate 2 dimensional lists, process a row, process a column, process a specific element
 - How to read, show and save images using the 3Dlist representing an image as provided in the cmpt120images module
 - How to extract and/or change the color as RGB and/or individual color components of a pixel using a 3DList representing an image as provided in the cmpt120images module

Broad Intro to Computer Vision

Many applications:

- captioning photos
- facial recognition (unlock your phone)
- filters to make your photos look cool
- medical applications
- lots of experts at SFU!

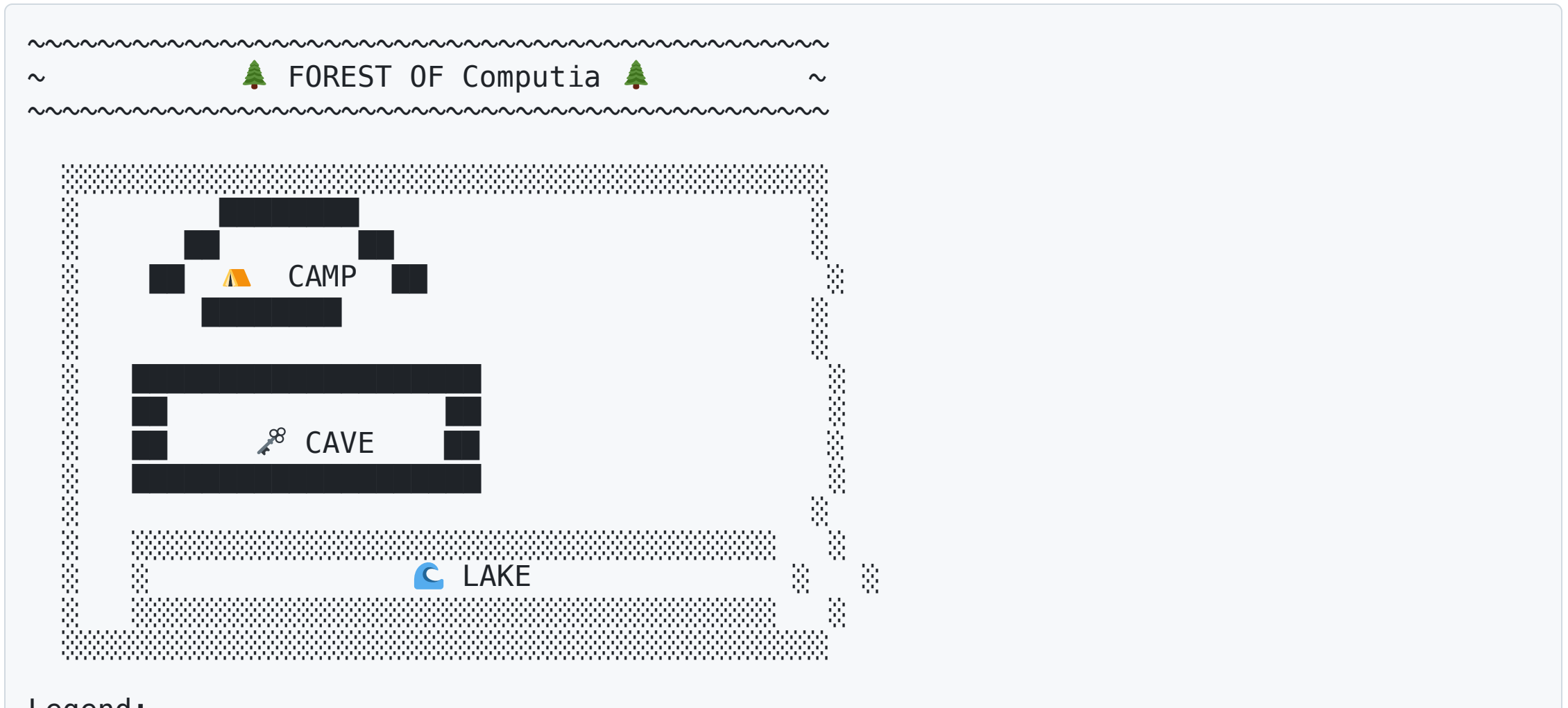
What is an image?

Simple question: "What is an image?"

- Philosophically?
- In analog media?
- In digital media?

A "Map" in Terminal

What if we wanted to make a game in our terminal?



A "Map" in Terminal

Using mostly zeros and 1s?

[illegible]

Legend:

1 = Wall/Boundary

$\emptyset$ = Open path

As actual python code

```
map_grid = [  
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],  
    [1, 0, 0, 0, 0, 1, 0, 0, 0, 1],  
    [1, 0, 1, 1, 0, 1, 0, 1, 0, 1],  
    [1, 0, 0, 1, 0, 0, 0, 1, 0, 1],  
    [1, 1, 0, 1, 1, 1, 0, 1, 0, 1],  
    [1, 0, 0, 0, 0, 0, 0, 1, 0, 1],  
    [1, 0, 1, 1, 1, 1, 0, 1, 0, 1],  
    [1, 0, 0, 0, 0, 1, 0, 0, 2, 1],  
    [1, 1, 1, 1, 0, 1, 1, 1, 1, 1],  
]  
  
# Display the map  
for row in map_grid:  
    print(" ".join(str(cell) for cell in row))
```

As Python code, but RBG triplets

```
map_grid_rgb = [
    [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [0, 0, 0], [255, 255, 255], [255, 255, 255], [255, 255, 255], [0, 0, 0]],
    [[0, 0, 0], [255, 255, 255], [0, 0, 0], [0, 0, 0], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0]],
    [[0, 0, 0], [255, 255, 255], [255, 255, 255], [0, 0, 0], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [255, 255, 255], [0, 0, 0], [0, 0, 0], [0, 0, 0], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0]],
    [[0, 0, 0], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0]],
    [[0, 0, 0], [255, 255, 255], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0], [255, 255, 255], [0, 0, 0]],
    [[0, 0, 0], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 255, 255], [0, 0, 0], [255, 255, 255], [255, 255, 255], [255, 255, 255], [255, 215, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [255, 255, 255], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
]

# Display the map in a human-readable way
for row in map_grid_rgb:
    print(" ".join(str(cell) for cell in row))
```

Ok, this is obviously getting a bit ridiculous.

If you really had to, I'd bet you could "complete this maze"...

but let's actually visualize it!

Motivating Task: Green Screen

Today's goal:

write a program that can merge a green screen image with another background, using the following concepts:

- Colours in RGB space
- Pixel representations of images in 2D arrays, e.g. `image[row][col]`
- Functions that return values
- Modules

Green Screen

What's a green screen?

https://en.wikipedia.org/wiki/Chroma_key

Prerequisites: Downloading images

We need to to download some files.

- `cmpt120image.py`
- `kid-green.jpg`
- `beach.jpg`

Prerequisites: Installing a new Python library

And we need to install a new Python library, `pygame`

This may be a challenge!

Using cmpt120image.py and pygame

Ok, time to import an existing image.

- Download pygame:
 - Try: `pip3 install pygame`
 - Didn't work? try `pip install pygame`
 - Didn't work? try `python3 -m pip install -U pygame==2.6.0`
 - Didn't work? Office hours

3D List

We're going to use the list-of-lists-of-3-item-list approach we saw above

(In Learning Outcomes, this is called 3D List)

- outermost list = rows
- second layer list = columns
- innermost list = RGB value

```
image = [  
    [ [0,0,0]  [255,255,255] ]  
    [ [0,0,0], [255,255,255] ]  
]
```

RGB reminders

Important (testable!) facts about RGB

- 3 integers between 0 and 255
- requires 2 digits in hexadecimal (between 0 - F)
- A is 10, B is 11, C is 12, D is 13, E is 14, F is 15
- only 16 options in hexadecimal so we just need 4 bits per character

```
FF -> 15 15 -> 1111 1111  
5A -> 5 10 -> 0011 0110
```

One single color (e.g. "light red") thus uses up $4 * 2 * 3 = 24$ bits

Conventions about rows and columns (120 and beyond)

- Should row come first or column come first? It's just convention.
- in other words: just another choice someone made in implementing a language / software / discipline!
- like the default arguments of our Python methods, the choice of which Python keywords are "special"
- Somebody just liked the words "for", "if", etc.
- In linear algebra $A_{i,j}$ conventionally means the i-th row, j-th column of matrix A

Example

$A_{1,1}, A_{1,2}, A_{1,3}$

$A_{2,1}, A_{2,2}, A_{2,3}$

$A_{3,1}, A_{3,2}, A_{3,3}$

See e.g.

[https://en.wikipedia.org/wiki/Matrix_\(mathematics\)](https://en.wikipedia.org/wiki/Matrix_(mathematics))

But compare with

https://en.wikipedia.org/wiki/Cartesian_coordinate_system

Where $\langle 1, 2 \rangle$ means go 1 unit right on the x-axis, then 2 units up.

Our convention:

- start from the top left
- first index for rows
- second index for columns
- Start with ZERO

This is a nice convention because it maps to how we loop through a matrix X, say:

1 2

3 4

```
X = [[1,2], [3,4]]  
  
for row in range(len(X)):  
    for column in range(len(X[0])):  
        print(X[row][column], end='')  
    print()
```

Write the indices!

Just like string indexing problems, if you're worried about making a mistake on paper questions
just write the indices!

```
A =  
1 2  
3 4
```

```
    0 1  
0 1 2  
1 3 4
```

Learn these conventions, but prepare for others

But ultimately, we are just somewhat-arbitrarily choosing to say

“Let’s represent our image as a list of lists...

Each of the individual lists will correspond to a row”

We could have each list represent a column. We could use one-indexing. Etc!

Why harp on this?

- For everyone: help avoid confusion and errors with rows and columns
- For those who want to do more linear algebra, computer vision, etc. – it may be of interest!

Height and width

Height and width of of an image use this list-of-lists (black and white) or list-of-list-of-lists (RGB) approach?

How to get height?

How to get width?

Try it!

- Right now, try a quick test in your terminal
- First, define a grid / matrix of integer values, using a list of lists
 - E.g., `image = [[1,2], [3,4]]`
- Then, try access each of the individual values
 - `image[0][0], image[0][1]`
- Then, try editing one of individual values

Starter code for green screen

In your new script, here's our starting code:

```
import cmpt120image as img_module
img_module.init()

img = cmpt120image.get_image('chatgpt_raccoon.jpg')
print(img[0][0])
```

A single pixel?

What's in `img[0][0]` ?

If time: Brief intermission, look at some example RGB color values.

Reminder of our goal:

Write a program that can merge a green screen image with another background, using the following concepts:

- Colours in RGB space
- Pixel representations of images in 2D arrays, e.g. `image[row][col]`
- Functions that return values
- Modules

How to merge a green screen image?

To merge a green screen image, we probably need to do something with all the green pixels?

What function might we want, to start?

is_green function

Complete the `is_green` function in Python.

It should return True if the red, green and blue channels are all within 30 of 0, 255, and 0 respectively.

```
def is_green(r,g,b):  
    """  
    Return true if the rgb value are within 30 of the color <0,255,0>  
    in other words, the r value can be 0-30, the g value 225-255, and the b value 0-30  
    input:  
    r - the red value (0-255)  
    g - the green value (0-255)  
    b - the blue value (0-255)  
    return True or False  
    """
```

```
def is_green(r,g,b):  
    """
```

Return true if the rgb value are within 30 of the color <0,255,0>

in other words, the r value can be 0-30, the g value 225-255, and the b value 0-30

input:

 r – the red value (0-255)

 g – the green value (0-255)

 b – the blue value (0-255)

return True or False

```
    """
```

```
    return r < 30 and g > 225 and b < 30
```

Note that

```
return r < 30 and g > 255 and b < 30
```

is a convenient way to return a boolean expression without actually using `if` statements

(Learning Outcome!)

Different arguments

How about rewriting our function so that it takes as arguments

- the entire image
- x coordinate
- y coordinate

```
def is_green(img, x ,y):  
    """
```

Return true if the rgb value are within 30 of the color <0,255,0>

in other words, the r value can be 0-30, the g value 225-255, and the b value 0-30

input:

img - the image

x - x coordinate

y - y coordinate

return True or False

```
    """
```

```
    pixel = img[x][y]
```

```
    r = pixel[0]
```

```
    g = pixel[1]
```

```
    b = pixel[2]
```

```
    return r < 30 and g > 255 and b < 30
```

The "Best" Arguments to accept?

Question: Which is better?

```
def is_green(img, x ,y):
```

vs

```
def is_green(r,g,b):
```

Multiple assignment

Advanced tip: Python has a nice convenient ("ergonomic!") feature to put elements in a list (if you know it's length) into variables

called "tuple unpacking" or "multiple assignment"

Can save some lines in our above function. This is optional.

```
r, g, b = img[x][y]
```

Review questions (Slide 1)

1. Name 2 applications areas of computer vision
2. If given an image in variable named `awesome_image`, how do we get the pixel that's pixels down, 8 to the right?
3. What happens if we define a variable in a function and then try to use it in the global scope?
4. What happens if we define a variable in a for loop and then try to use it after the for loop?

Review questions (Slide 2)

5. Which colour is `[0,0,50]` closet to? (dark / light + red / green / blue)
6. In a blank Python file, initialize a list of lists of lists that gives a small, 2x2 4 pixel image with red in top left, green in top right, blue in bottom left, and black in bottom right.

Review questions (Slide 3)

7. Look at this list

```
img = [ [[10,20,30], [15,20,200]],  
        [[25,0,98], [5,0,0]],  
        [[60,200,0], [0,0,200]] ]
```

7a. How many pixels does it have?

7b. What's the height?

7c. What's the width?

7d. How many mostly blue pixels?

7e. How many pixels with no red?

Nested loops and images

We can use nested loops to initialize an image

```
for row in range(100):  
    for col in range(100):  
        img[row][col] = [0,0,0]
```

Traversing image with nested loops

We can also traverse our image this way.

Let's put our snippets together into a single file.

```
for row in range(100):  
    for col in range(100):  
        img[row][col] = [0,0,0]
```

Try to write a file!

Ok! We can give it a first go.

Any problems?

Two options for debugging:

1. Think really hard about it (sometimes a good idea; when you ask me questions I sometimes start with this approach. But if the code is long or there's a long line of people I might ask you to start using "print debugging")
2. Look closely at the output that doesn't meet expectations

Reorganizing our code into a module

- Put our function in a separate file
- Now we can `import xyz`
- Let's call it "green_checker" -- the one stop shop for all your is-my-pixel-green needs
- Something to think on: how could we extend this module to handle many colours?

- Short names: you can rename modules as you import them using `as`
 - `import pandas as pd`
 - `import mycoolcoloursmodule as colour`

So if you want to give your module a long name: `super_awesome_green_checker`, you might do

```
import super_awesome_green_checker as gc
```

then run

```
gc.is_green
```

What's in cmpt120image.py?

It's actually just a "wrapper" for `pygame` .

- pygame is a "package" -- a collection of modules
- Anyone heard of a "GPT wrapper?"

More questions

- What is the name of a module relative to the filename?
- What can nested loops be used for in the context of image processing?
- What is the difference between a package and a module?

What does answer contain after the code is run below?

```
def special(numbers):  
    for x in numbers:  
        for y in numbers:  
            if x < y:  
                return x+y  
  
answer = special([3,1,2])
```

- a) How many bytes does the string "Aha!" need to be coded using the ASCII code?
- b) Write 32 using 1-byte binary.

What will be the result of executing this code? If there is some problem, how can it be fixed?

```
def function_a(p):  
    x = p + 100  
    return x  
result = function_a(1)  
print(result)
```

What will be the result of executing this code? If there is some problem, how can it be fixed?

```
def function_b(p):  
    p = p + 100  
    return p  
arg = 1  
print("arg before function",arg)  
value = function_b(arg)  
print(value)  
print("arg after function",arg)
```

What will be the result of executing this code?

```
def functionC(p):  
    x = p + 100  
    print(x)  
arg = 1  
value = functionC(arg)  
print(value)
```

What does the user see when this code is executed?

```
def function_e(lst):
    found = "no"
    for word in lst:
        if "a" in word:
            return("the word " + word + " has an 'a'")
            found = "yes"
        if "b" in word:
            return("the word " + word + " has a 'b'")
            found = "yes"
    if found == "yes":
        result = "I found at least a word with an 'a' or at least a 'b'"
    else:
        result = "I did not find any word in the list with 'a' nor with 'b'"
    return result

words = ["perfect", "great", "absolutely"]
messg = function_e(words)
print(messg)
```

What does the user see when this code is executed?

```
def add_stars(word):  
    result = word + "**"  
    return result  
  
lst = ["I", "am", "very", "happy"]  
order = [2, 3, 0, 1]  
message = ""  
for i in range(len(order)):  
    starred = addStars(lst[order[i]])  
    message = message + starred  
  
print(message)
```

Brief foray into memory

- On our hard drive, we have RAM
- RAM = Random Access Memory (fast)
- Each object in our program has an address in memory
 - Variables references / points to that address
 - we can use python `id` to get that address, e.g.

```
x = 5 # means there's a "bucket/slot" in our RAM somewhere that is holding the value 5
print(id(x)) # prints out that id
```

Here's where things get a little weird with list

if we do this:

```
a = [1,2,3]  
b = a
```

B is an alias of a. Points to the same address. How to check?

```
id(a) == id(b)
```

But if we do this:

```
a = [1,2,3]  
c = a[:]  
d = list(c)
```

Both c and d are copies of a. New address, same content.

Why might this matter?

An alias doesn't take extra space (pro) but if we edit the original, we also edit the alias (sometimes good, sometimes bad)

A copy takes extra space, but we can edit them separately

To the Python code visualizer!

<https://pythontutor.com/visualize.html#mode=edit>

One concrete thing to know: Append is slightly more efficient than list concat

List concat makes a copy under the hood, uses up more of our "memory buckets"

List .append is a bit more efficient

Another concrete thing to know (classic Python interview gotcha!): Lists (and mutable objects) are passed by reference

Important Python fact: lists are passed by references

So are dicts and other "mutable objects". For purposes of 120, lists are our main mutable object right now.

Most variables (immutables like int, float, string) are not passed by reference, they are passed by assignment!

What about lists of lists?

The list slicing and `list()` ways of cloning lists only works for 1-dimensional lists. For lists that contain sublists, only a shallow copy will take place of the outer list. The elements in the sublists will still point to the original objects.

In the final project, you will need to be aware of this.

If you do not wish to change the original image, create a new image using `get_black_image()`, for example.

Some 2 cents

- Sometimes efficiency is key (some CMPT classes, some coding interviews)
- But sometimes readability / convenience is key!
- Learning to balance is an art.

I use list concatenation sometimes when it's not optimal in my personal research code...

But I might be a bit more thoughtful if I know my code will be run "in prod"

More 2 cents

In general, this stuff isn't a common issue in Python – you can often not think about references and get away with it.

Some Python intros may not talk about it at all.

Most common place where this matters is when you're mutating multi-level data structures (list of lists, dicts of dicts)

When it matters, it can matter a lot – so knowing how to identify, test for, and fix alias / cloning related issues can make you stand out as a programmer!

In other CMPT courses, you'll use languages like C++ that are much more "explicit" about memory

One Slide Summary

Aliases

- Variables can refer to the same object.
- Example: `a = [1, 2, 3]; b = a`
- Changing b affects a, since both are aliases of the same list.

Copies

- Shallow Copy: `new_list = old_list.copy()` or `new_list = old_list[:]`
 - Creates a new object but doesn't copy nested objects.
- Deep Copy: `import copy; new_list = copy.deepcopy(old_list)`
 - Copies all nested objects, creating a completely independent clone.

Pass by Reference

- Python uses "pass-by-assignment."

Binary Search

[Interactive Binary Search Demo](#)

A Real-life Example

Let's say you want to sort Pokémon cards.
You could sort by HP — how might you do it?

Swapping Pattern

Let's say we want to swap the values at two different spots in a list.

```
temp ← a  
a ← b  
b ← temp
```

Selection Sort

Algorithm:

- For every element in the list:
 - Find the smallest element in the rest of the list
 - Swap the current element with that smallest element

[Selection Sort Reference](#)

Timing Programs

You can use the `time` module to measure how long your program takes.

```
import time
start = time.time()
# your code
end = time.time()
print(end - start)
```

Visualization of Sorting Algorithms

Source: [GitHub - vbohush/SortingAlgorithmAnimations](#)

YouTube: [Sorting Algorithm Visualization](#)

Goodness of an Algorithm

1. **Correctness** — It works.

2. **Desirable qualities:**

- Readable and easy to debug
- Clear and structured
- Robust and maintainable
- Efficient in time and space

Two Ways to Examine "Goodness"

- **Time Complexity** – efficiency of execution
- **Space Complexity** – efficiency of memory use

Efficiency is key for large-scale problems.

Complexity

Amount of time or space required to run an algorithm.

We focus on **time complexity** in this course.

Measuring Time Complexity

We often measure:

- Execution time for a given problem
- Or, number of operations performed (hardware-independent)

Order of an Algorithm

Represents how the number of operations grows with input size n .

Examples: $O(n)$, $O(n^2)$, $O(\log n)$

Standard Reference Functions

Category	Function
Constant	1
Logarithmic	$\log_2(n)$
Linear	n
Linearithmic	$n \log_2(n)$
Quadratic	n^2
Cubic	n^3
Exponential	$a^n, a > 1$

[More on Big O](#)

Example: Calculating Time Complexity

```
x = 0
y = 10
x += 1
for i in range(n):
    x += y
    y += 1000
```

→ $O(n)$ complexity.

Example 2: Nested Loops

```
count = 0
for i in range(n):
    count += 10
    for j in range(n):
        count += j
```

→ $O(n^2)$ complexity.

Best, Worst, and Average Case

- **Best:** fastest scenario
- **Worst:** slowest scenario
- **Average:** expected scenario

Example: Linear Search

- Best: Found at first element
- Worst: Found at last element or not found

Linear Search Complexity

Case	Complexity
Best	$O(1)$
Worst	$O(n)$

Logarithmic Complexity

Each iteration halves the problem size $\rightarrow O(\log n)$

Examples:

- Binary search
- Divide and conquer algorithms

Binary Search Analysis

Each split halves the data until 1 element remains.

Height of the tree = $\log_2 n$ comparisons (worst case).

Selection Sort Complexity

Selection sort compares:

$$n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2 \approx O(n^2)$$

Applies to both best and worst cases.

Comparing Algorithms

Algorithm	Big O
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Selection Sort	$O(n^2)$

Is Sorting Worth It?

Binary search requires sorted data, but sorting costs $O(n^2)$.

Sorting first makes sense when:

- You search many times after sorting.

When n Gets Larger

Algorithm	Big O	Example (n=1000)
Constant	$O(1)$	1
Linear	$O(n)$	1000
Quadratic	$O(n^2)$	1,000,000
Logarithmic	$O(\log n)$	~10

Introducing Merge Sort

Much faster than Selection Sort.

Visualization

Merge Sort Concept

1. Split list until each has one element
2. Merge sorted sublists

Merge Sort Pseudocode

```
def mergeSort(alist):  
    if len(alist) > 1:  
        left = mergeSort(alist[:mid])  
        right = mergeSort(alist[mid:])  
        return merge(left, right)  
    else:  
        return alist
```

Merge Sort Complexity

- Splits → $\log n$
- Merges → n
- **Total: $O(n \log n)$**

Uses extra space for merging.

Review

Algorithm	Worst Case Complexity
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Selection Sort	$O(n^2)$
Merge Sort	$O(n \log n)$

Final Thought

If two algorithms are $O(n)$ and $O(n \log n)$,
and you have a **large dataset**,
choose the $O(n)$ algorithm.

Week 12 Recursion

This folder includes some interactive content we'll work through together during Week 12!

We'll spend some time trying to code key recursive functions and live code them together.

This deck has the SAME content as the readme file in

`other_content/week12_recursion`. I will recommend you open that file during lecture, and I'll project this just for slightly better lecture viewing experience.

Exercise 1: Review tree drawing

First, take a look at `draw_tree.py` in this directory. We're going to talk through how and way this code works. You should copy and paste this into a file on your local machine.

Trying running it, then try modifying the *initial function call* so that the tree:

1. Has even more branches
2. Has fewer branches
3. Hits the base case immediately

Exercise 2: Recursive Factorial

Problem: Write a recursive function that returns the factorial of a number. The factorial of n is equal to n times the factorial of $n-1$.

Start with a single parameter function definition: `def factorial(n):`

Examples:

- $0! = 1$ (important: the factorial of 0 is equal to 1.)
- $1! = 1$
- $2! = 2 * 1 = 2 * 1!$
- $3! = 3 * 2 * 1 = 3 * 2!$
- so, `factorial(4) # -> 24`

► Hint 1: Base Case

► Hint 2: Recursive Case

► Solution

Exercise 3: Sums with recursion

Exercise 3a: Summing all integers from 1 to n

Write a recursive function in Python, `recursive_sum`, that calculates the sum of all integers from 1 up to a given positive integer `n`.

Start with a single parameter function definition: `def recursive_sum(n):`

Examples:

- `recursive_sum(4)` # Output: 9 (1 + 2 + 3 + 4)
- `recursive_sum(5)` # Output: 15 (1 + 2 + 3 + 4 + 5)

► Hint 1: Base Case

► Hint 2: Recursive Case

► Solution

Exercise 3b: Summing all integers in a list

Problem: Write a recursive function in Python, `recursive_sum_list`, that calculates the sum of all integers in a given list.

Start with a single parameter function definition: `def recursive_sum_list(numbers):`

Examples:

- `recursive_sum_list([1, 2, 3, 4])` # Output: 10
- `recursive_sum_list([5, 7, 3])` # Output: 15

► Hint 1: Base Case

► Hint 2: Recursive Case

► Hint 3: List Indexing

► Solution

Exercise 4: Reverse a string

Write a recursive function in Python, `recursive_reverse`, that reverses a given string. Start with a single parameter function definition: `def recursive_reverse(s):`

Example:

- `recursive_reverse("hello")` # Output: "olleh"
- `recursive_reverse("abc")` # Output: "cba"

► Hint 1: Base Case

► Hint 2: Recursive Case

► Hint 3: String Indexing

► Solution

Exercise 5: Palindrome checker

Write a recursive function in Python, `is_palindrome`, that checks if a given string is a palindrome (reads the same forward and backward). Start with a single parameter function definition: `def is_palindrome(s):`

Examples:

- `is_palindrome("racecar")` # Output: True
- `is_palindrome("hello")` # Output: False

► Hint 1: Base Case

► Hint 2: Recursive Case

► Hint 3: String Indexing

► Solution

Exercise 6: Challenge!

"Nested List Sum"

Problem: Write a recursive function in Python, `nested_sum`, that calculates the sum of all integers in a list. However, this time, some elements of the list might be nested lists themselves. The function should recursively process these nested lists to calculate the total sum.

You can use `isinstance(item, int)` to check an item is an integer and `isinstance(item, list)` to check if an item is a list.

Start with a single parameter function definition: `def nested_sum(data):`

Examples:

- `nested_sum([1, 2, [3, 4], 5])` # Output: 15
- `nested_sum([[1, 2, [3]], 4, [5, [6]]])` # Output: 21

► Solution

Exercise 7: Count Vowels in a String

Problem: Write a recursive function, `count_vowels`, that counts the number of vowels (a, e, i, o, u) in a given string. The function should be case-insensitive, so both uppercase and lowercase vowels are counted.

Start with a single parameter function definition: `def count_vowels(s):`

Examples:

- `count_vowels("hello") # Output: 2`
- `count_vowels("Recursion is fun!") # Output: 6`
- `count_vowels("xyz") # Output: 0`

► Hint 1: Base Case

► Hint 2: Recursive Case

► Solution

Binary Search: A Fast Way to Find Things

Agenda / About this deck

First, finish our recursion activity.

Then, run through this deck very quickly. This is just focused on the code implementation of binary search (lots of overlap with Runestone).

Then, depending on time, we'll talk more conceptually about search, but also just work on project.

Introduction to Searching

- **Searching:** Imagine looking for a word in a dictionary. If you check one word at a time from the start, it would take ages for a long dictionary. Binary Search makes this faster.
- **Key Idea:** Divide the search space in half each time.

Linear Search vs. Binary Search

- **Linear Search:** $O(n)$
 - Check each element one-by-one.
 - Useful when data is unsorted.
- **Binary Search:** $O(\log n)$
 - Faster, but requires sorted data.
 - Each time, cut the search space in half.

Interactive Demo: Imagine finding a word in a dictionary. How many words do you skip if you flip halfway each time?

Binary Search Intuition

- **Sorted Array Example:** [1, 3, 5, 7, 9, 11, 13]
 - Target: Find 9.
 - **Step 1:** Start with middle element (7).
 - **Step 2:** Since $9 > 7$, ignore left half.
 - **Step 3:** Now search [9, 11, 13]. Middle is 11.
 - **Step 4:** Since $9 < 11$, ignore right half. Find 9.

Visualization

- **Divide-and-Conquer:**
 - Use visuals to demonstrate the division of the array.
 - Show the array shrinking in size each time.
- **Pseudocode:**

```
low = 0
high = len(array) - 1
while low <= high:
    mid = (low + high) // 2
    if array[mid] == target:
        return mid
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return -1
```

Why Does It Work?

- **Core Concept:** Every decision eliminates half of the possible choices.
- **Comparison to Real Life:** Imagine finding a page in a book without an index. Flip to the middle until you narrow it down.

Practice Problem

- **Problem:** Find the number 23 in the sorted list [5, 12, 15, 23, 27, 30, 35].
- **Thinking out loud:** Can try to write out / verbalize each step.
 - "Is 23 greater or less than the middle element?"
 - Keep cutting until found.

Common Mistakes

- **Off-by-One Errors:** Carefully manage `low` and `high` boundaries.
- **Infinite Loops:** Ensure condition `low <= high`.

Tip: Debugging exercises can help solidify this. Run a failing example and walk through the fix.

Example mistake 1: Incorrect Mid Calculation

```
array = [1, 3, 5, 7, 9, 11, 13]
target = 9

low = 0
high = len(array) - 1
while low <= high:
    mid = (low + high) / 2 # Mistake: Using `/` instead of `//` for integer division
    if array[mid] == target:
        print(f"Found target {target} at index {mid}")
        break
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Target not found")
```

Mistake 2: Off-by-One Error

```
# Mistake 2: Off-by-One Error
array = [1, 3, 5, 7, 9, 11, 13]
target = 9

low = 0
high = len(array) # Mistake: Should be `len(array) - 1`
while low <= high:
    mid = (low + high) // 2
    if array[mid] == target:
        print(f"Found target {target} at index {mid}")
        break
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Target not found")
```

Example Mistake 3: Infinite Loop Condition

```
array = [1, 3, 5, 7, 9, 11, 13]
target = 9

low = 0
high = len(array) - 1
while low < high: # Mistake: Should be `low <= high`
    mid = (low + high) // 2
    if array[mid] == target:
        print(f"Found target {target} at index {mid}")
        break
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Target not found")
```

Fully Worked Example of Binary Search

```
def binary_search(array, target):
    low = 0
    high = len(array) - 1

    while low <= high:
        mid = (low + high) // 2
        # Debug statement to show the current state of low, high, and mid
        print(f"Low: {low}, High: {high}, Mid: {mid}, Mid Value: {array[mid]}")

        if array[mid] == target:
            return mid
        elif array[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1 # Target not found

# Example usage
array = [1, 3, 5, 7, 9, 11, 13]
target = 9
result = binary_search(array, target)
if result != -1:
    print(f"Target {target} found at index {result}")
else:
    print("Target not found")
```

Binary Search Applications

- **Applications:**
 - Finding an element in a sorted array.
 - Searching in a phonebook.
 - Problems like "Guess the Number" (using minimum attempts).

Conclusion & Recap

- **Summary:** Binary Search cuts the search space in half each time, leading to a time complexity of $O(\log n)$.
- **Key Question:** Why does Binary Search need a sorted array?

Engagement: Think of any real-life activities where a binary search-like strategy is used?

<https://www.cs.usfca.edu/~galles/visualization/Search.html>

The Big Review Deck



From Priscilla Du Preez @ <https://unsplash.com/photos/scenery-of-bridge-in-front-of-silhouette-of-mountain-Vm0nC-VKFTc>

Week 1: Introduction to CS and Problem Solving

- Know that CS is problem-solving
- Problem solving by subdividing tasks
- Characteristics of an algorithm
- Plagiarism rules for programming
- What is pseudocode?
- Write Python comments, program header block info
- Output "Hello World" using print()
- Programming languages vs. natural languages
- Use IDLE and an IDE
- Submit code as .zip or .py file

Week 2: Basics of Python Programming

- Design/plan algorithms using pseudocode/comments
- Input from terminal to a variable or without saving
- Assign a value to a variable
- Output a string variable, string concatenation
- Variable naming rules
- Data types introduction (focus on String)
- Lists and `random.choice()` (including imports)
- Purpose of modules, imports at top
- If/elif/else, logical operators (`and` , `or`)
- Comparison operators in conditional statements
- Boolean expressions, testing programs interactively

Week 3: Strings, Lists, and Conditionals

- String methods: `strip()`, `lower()`, `upper()`
- Identify String data type
- Test methods using REPL
- Use `in` keyword with strings and lists
- Create a list from variables/user inputs
- Loop over list elements with `for`
- Understand `range()` function and index variable
- Integer type and conversion to String
- Concatenation rules, nested conditionals
- Error types: syntax vs. semantic
- Method chaining with `.`

Week 4: Loops and Arithmetic

- Use `range()` with variables - Loop to reduce code duplication - Integers, floats, initialization - Accumulator pattern, `+=` shortcut - Get list length, convert input to Integer
- Division types, arithmetic operations - Print floats to given decimals

```
# Accumulator Pattern and Loop with Range
total = 0
for i in range(1, 6): # Loop from 1 to 5
    total += i # Accumulate the sum

print(f"Sum of numbers from 1 to 5 is: {total}")

# Basic arithmetic operations
div_result = 10 / 3 # Division (float)
print(f"Division result: {div_result:.2f}") # Print to 2 decimal places
```

Week 5: File I/O, Lists, and Comparisons

- Read lines from text files
- String splitting, list indexing/slicing
- Compare numbers and strings
- operator precedence (light treatment)
- Nested conditionals, nested loops
- find common elements between lists
- List concatenation, accumulation pattern for strings/lists
- Calculate max/min, coordination between lists by index

```
# Reading a File and Splitting Lines into Lists
with open("example.txt", "r") as file:
    for line in file:
        words = line.strip().split() # Split line into words
        print(words) # List of words
```

Week 7: Bits, Bytes, and Data Representation

- `len()` function, string immutability, loops by index
- Bits and bytes, converting bytes to bits
- Binary to decimal, hexadecimal goals
- ASCII and Unicode overview
- RGB storage in hexadecimal
- Important numbers: 1 byte = 8 bits, 8-bit per character for 8-bit character set, 6 hexadecimal digits in an RGB number, therefore 24 bits, because 6 digits x 4 bits per digit
- Different numbering systems (base-2, base-10, base-16) require different number of "symbols"

```
# Convert Bytes to Bits and Decimal to Binary
bytes_value = 4
```

Week 8: Turtle Graphics and Functions

- Use Turtle package for drawing (specific methods that are "testable" to be described in class)
- Read Turtle code and understand the purpose
- Create and call functions (with/without parameters)
- Loop variable as function argument
- Turtle coloring using RGB values
- Variable scope (function vs. global)

```
# Turtle Graphics Drawing a Square
import turtle

def draw_square(size):
    pen = turtle.Turtle()
    for _ in range(4):
        pen.forward(size)
```

Week 9: Functions and While Loops

- Define and use functions, `return` vs `print()`
- Effect of `return` in loops, handling `None`
- Identify appropriate use of `while` vs `for`
- Sentinel and multiple control variables in `while`
- Use `while` for input validation

```
# Function that Returns a Value and While Loop for Validation
def double_number(num):
    return num * 2

valid_input = False
while not valid_input:
    user_input = input("Enter a positive number: ")
    if user_input.isdigit() and int(user_input) > 0:
        valid_input = True
        result = double_number(int(user_input))
```

Week 10: Modules and Image Processing

- Create and use custom modules
- Module shortnames and relationship of modules to filesystem
- RGB colors and pixel manipulation
- Access/modify 2D/3D lists for images
- Using course-specific `cmpt120images` module for images
- Return Boolean expressions directly

```
# Custom Module Import and RGB Color Manipulation
import cmpt120images

image = cmpt120images.loadImage("example.png")
height = len(image)
width = len(image[0])

for y in range(height):
```

Week 11: Lists, Aliases, and Recursion

- Alias vs. copy of lists
- Function parameter and argument as aliases
- List modification inside functions
- Basic recursion, draw recursive patterns
- 3 characteristics of recursive function
- Write factorial, sum of list, reverse string using recursion
- Check if a string is a palindrome (recursive/iterative)

```
# Recursive Function to Calculate Factorial
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

Week 12: Search Algorithms

- Review recursion
- Write linear search functions (Boolean/index/indices)
- Recognize and code recursive binary search

```
# Binary Search Implementation
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

```
numbers = [1, 3, 5, 7, 9, 11]
index = binary_search(numbers, 7)
```

Week 13: Sorting and Complexity

- Swap list elements, selection sort
- Intermediate steps of selection sort
- Measure runtime with `time`
- Apply algorithms on data from a file
- Characteristics of a good algorithm
- Big O notation and complexity examples. 7 reference functions.
- Merge Sort approach and complexity
- Critical operations, best/worst case scenarios
- Analyze binary search and merge two lists

```
# Selection Sort Implementation
def selection_sort(arr):
    n = len(arr)
```